

Algorithmes des n -reines

Jorel Raphaël

Pour commencer

Le hasard en informatique est quelque chose de difficile à obtenir. Le “vrai” hasard n’existe pas vraiment dans cette science, mais il peut être approché. A quoi peut-il servir dans ce domaine ? Un algorithme peut-il avoir une solution en utilisant le hasard ? La réponse à cette seconde question est oui, comme nous allons le voir très vite, la première est l’objet de cette petite étude. Disons plutôt que cette dernière vise à montrer un exemple dans lequel le hasard peut être utile. Nous utiliserons pour cela le problème des n -reines et quatre algorithmes qui le résolvent, déterministes et/ou randomisés. Pour rappel, le terme “randomisé” désigne les algorithmes qui utilisent le hasard, tandis que le terme “déterministe” est attribué à ceux qui ne l’utilisent pas.

Nous nous intéressons donc ici à quatre algorithmes de résolutions des n -reines, algorithmes par permutations, backtracking, Las Vegas et un dernier hybride, mêlant les techniques du Las Vegas couplées au backtracking. Nous verrons d’abord quelques informations générales concernant ce problème, avant de montrer quelques figures comparant les différents algorithmes en terme d’efficacité en temps et en nombre de combinaisons testées dans les cas où l’on ne cherche qu’une seule solution et celui où nous les voulons toutes.

Pour rappel, le problème des n -reines consiste à placer n reines sur un plateau de $n \times n$ cases, avec comme objectif qu’aucune d’entre elles ne soient en prise d’une autre, c’est-à-dire ni sur une même ligne, une même colonne ou une même diagonale qu’une autre.

Combinaisons

Si l’on considère que deux reines ne peut être sur la même case, mais sur une même ligne, un même colonne, ou les deux, alors le nombre de possibilités est $A_{n^2}^n$, soit le nombre d’arrangements possibles au sens mathématique.

Ce qui, pour les valeurs $n = 8, 10, 12$, nous donne :

- $n = 8 \Rightarrow A_{64}^8 = 178462987637760$
- $n = 10 \Rightarrow A_{100}^{10} = 6.281565095552947e^{+19}$
- $n = 12 \Rightarrow A_{144}^{12} = 4.963380753290496e^{+25}$

Solutions

Le nombre de solutions pour les valeurs $n = 8, 10, 12$ sont :

- $n = 8 \Rightarrow 92$,
- $n = 10 \Rightarrow 724$,
- $n = 12 \Rightarrow 14200$.

Vu le nombre de solutions très important, je pense que toutes les énumérer serait un peu long et assez inutile (à mon sens).

Première solution

Voici donc maintenant quelques graphiques permettant de comparer chaque algorithme pour trouver la première solution en terme de temps et de nombres de combinaisons testées s’avérant ne pas être des solutions. Afin de comprendre quels sont les modes opératoires de chaque algorithme, je vais les expliquer brièvement. Je parlerais aussi très rapidement des techniques utilisées pour faire ces mesures.

Permutations (déterministe) L’algorithme commence avec les reines déjà placées sur le plateau, et les permute deux à deux, jusqu’à trouver les solutions du problème.

Backtracking (déterministe) En Français “retour sur trace”, cet algorithme place les reines une par une, en vérifiant qu’il le peut. Quand il est bloqué, il déplace les reines précédemment placées et recommence. C’est la manière la plus naturelle pour un humain de procéder.

Las Vegas (randomisé) Cet algorithme-ci se base sur le hasard. Les reines sont donc placées sur le plateau en suivant une loi uniforme, afin que chaque case ait autant de chance d’être choisie que les autres pour y mettre une reine. Si la combinaison finale n’est pas valide, on recommence le procédé jusqu’à trouver une solution.

Hybride (déterministe - randomisé) Un mélange de Las Vegas et de backtracking. L’algorithme de Las Vegas est utilisé pour placer au hasard les $k(\leq n)$ premières reines, puis la main est passée au backtracking pour terminer le travail. Seulement des fois, l’algorithme de Las Vegas pose ses k reines de manière à ce que l’algorithme de backtracking ne puisse pas trouver de solution. Plutôt que d’avorter la combinaison en cours, ce dernier revient sur les reines placées par son prédécesseur, jusqu’à trouver une solution.

Mesures J’utilise une implémentation en C de ces algorithmes. Pour la mesure du temps, je me sers des structures *timeval*, qui permettent la précision jusqu’à la microseconde, mais je me borne à la milliseconde car cela est inutile d’être plus précis (je dis cela car j’ai essayé d’abord avec les microsecondes et que cela n’a pas influencé les courbes). Pour la mesure du nombre de tests, j’utilise une variable globale que j’incrémente à chaque fois que je vérifie qu’une reine peut être placée, ou qu’une combinaison est valide (ou non).

Regardons d’abord les trois premiers algorithmes, à savoir permutation, backtracking et Las Vegas. L’algorithme hybride est gardé pour la fin, car il est intéressant de l’étudier en fonction du nombre de reines déjà placées. Regardons donc les temps nécessaires et le nombre de combinaisons testées avant de trouver la première solution, en fonction du nombre de reines.

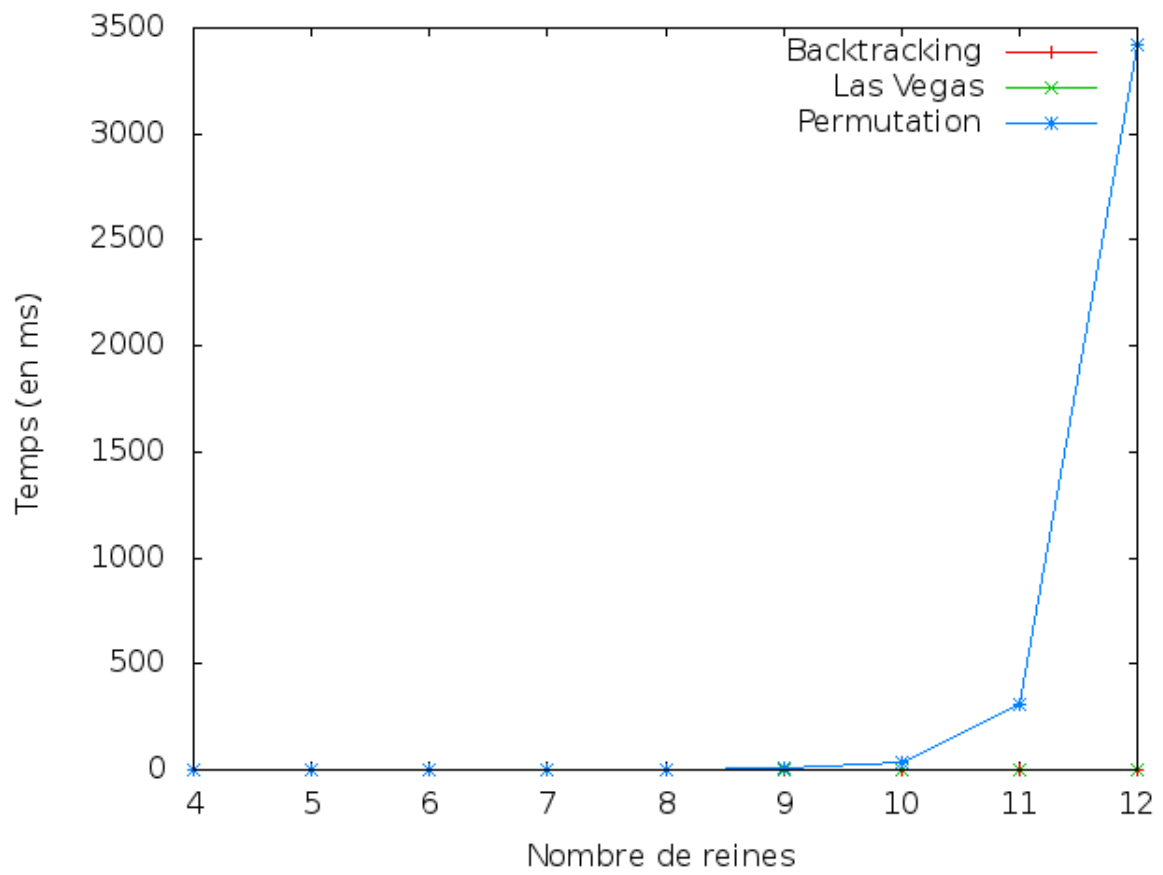


FIGURE 1 – Temps nécessaires à chaque algorithme (sauf hybride)

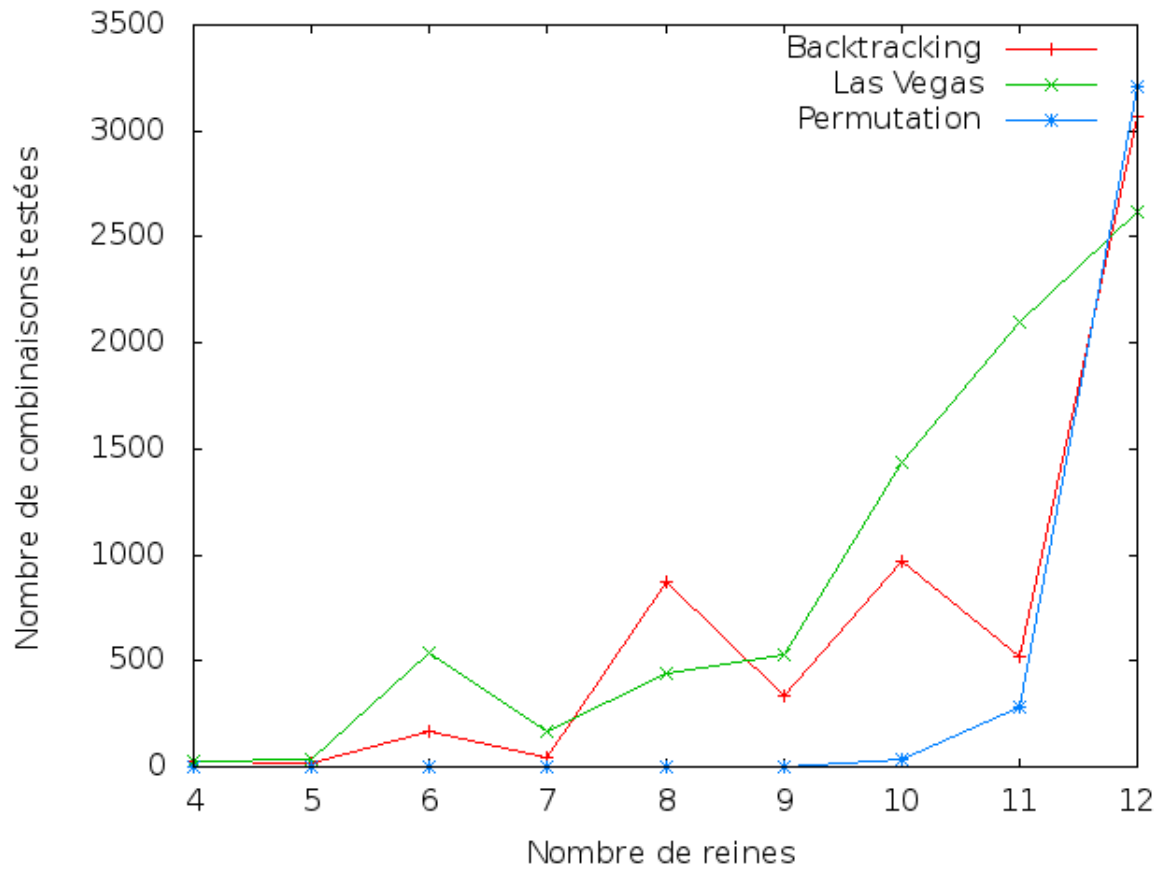


FIGURE 2 – Combinaisons testées par chaque algorithme (sauf hybride)

Nous observons que l'algorithme de permutation est beaucoup plus long à exécuter que les autres, donc pour des raisons matérielles, il est difficile d'aller plus loin que 12 reines. C'est pourquoi je propose maintenant de regarder jusqu'à 16 reines avec les deux autres pour voir lequel est le plus efficace en fonction du nombre de reines.

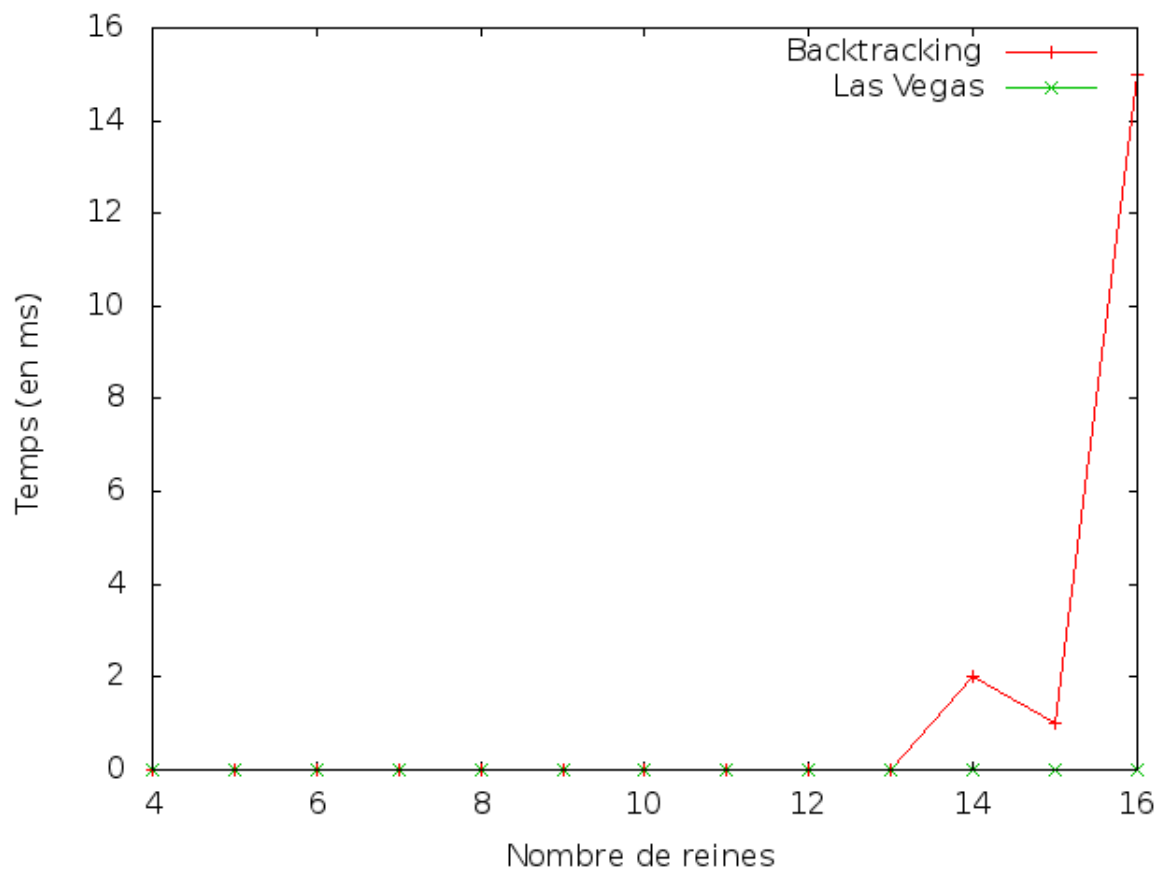


FIGURE 3 – Temps nécessaires au backtracking et à Las Vegas

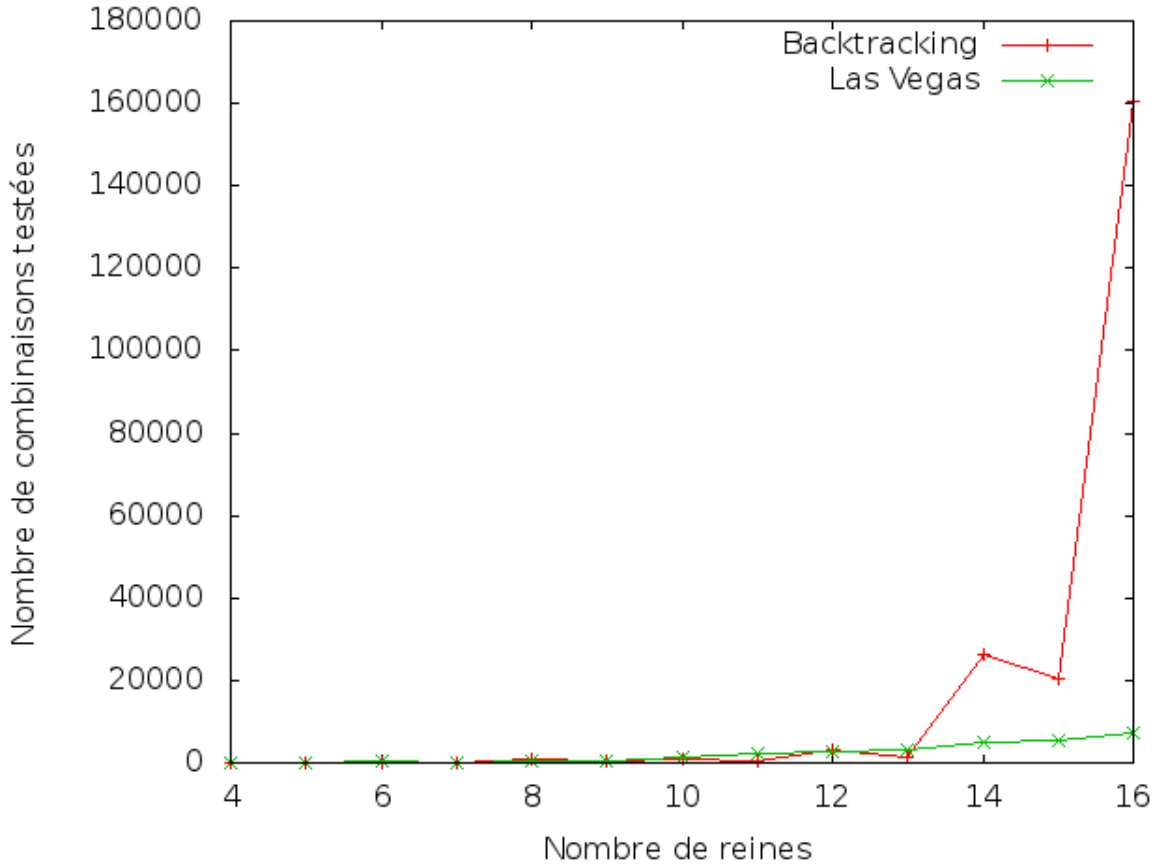


FIGURE 4 – Combinaisons testées par le backtracking et Las Vegas

Nous remarquons que l'algorithme de Las Vegas reste le plus performant sur les deux points étudiés (vitesse et nombre de combinaisons), mais également que cet algorithme teste plus de combinaisons jusqu'à $n = 13$. D'ailleurs c'est à partir de cette valeur que l'algorithme de backtracking commence à prendre plus de temps pour trouver la solution que son concurrent.

Et maintenant, intéressons-nous à l'algorithme hybride. Comme dit précédemment, il est plus pertinent de le traiter à part, car le nombre k de reines placées par l'algorithme de Las Vegas influe beaucoup sur la vitesse de l'algorithme hybride. Dans les cas des algorithmes randomisés, le hasard peut influencer beaucoup sur leur vitesse d'exécution. Il n'est pas rare d'exécuter deux fois un algorithme et d'obtenir des performances très éloignées en terme de temps. Pour avoir des résultats représentatifs, il faut faire une moyenne de plusieurs exécutions de ces algorithmes. J'ai indiqué ce nombre dans la prochaine figure, car la moyenne faisait descendre tous les temps à 0, la courbe n'était donc pas intéressante.

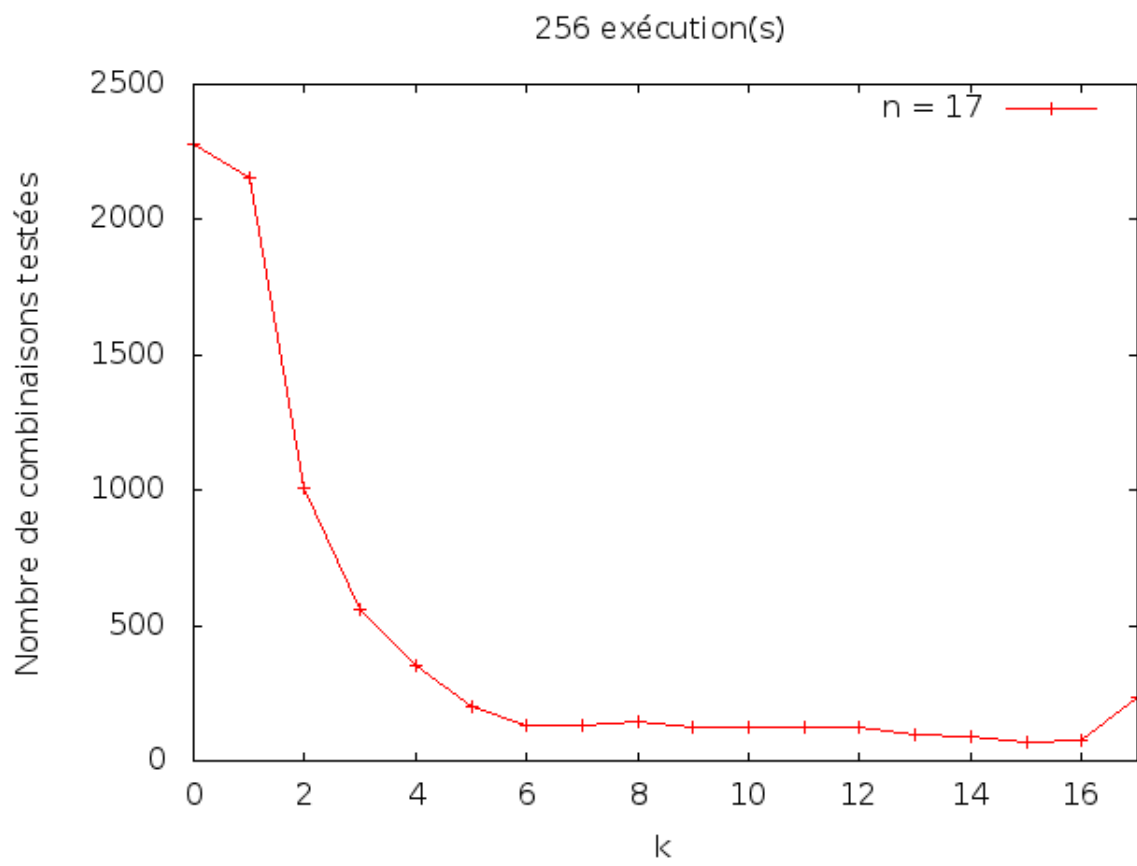


FIGURE 5 – Temps nécessaire à l’algorithme hybride

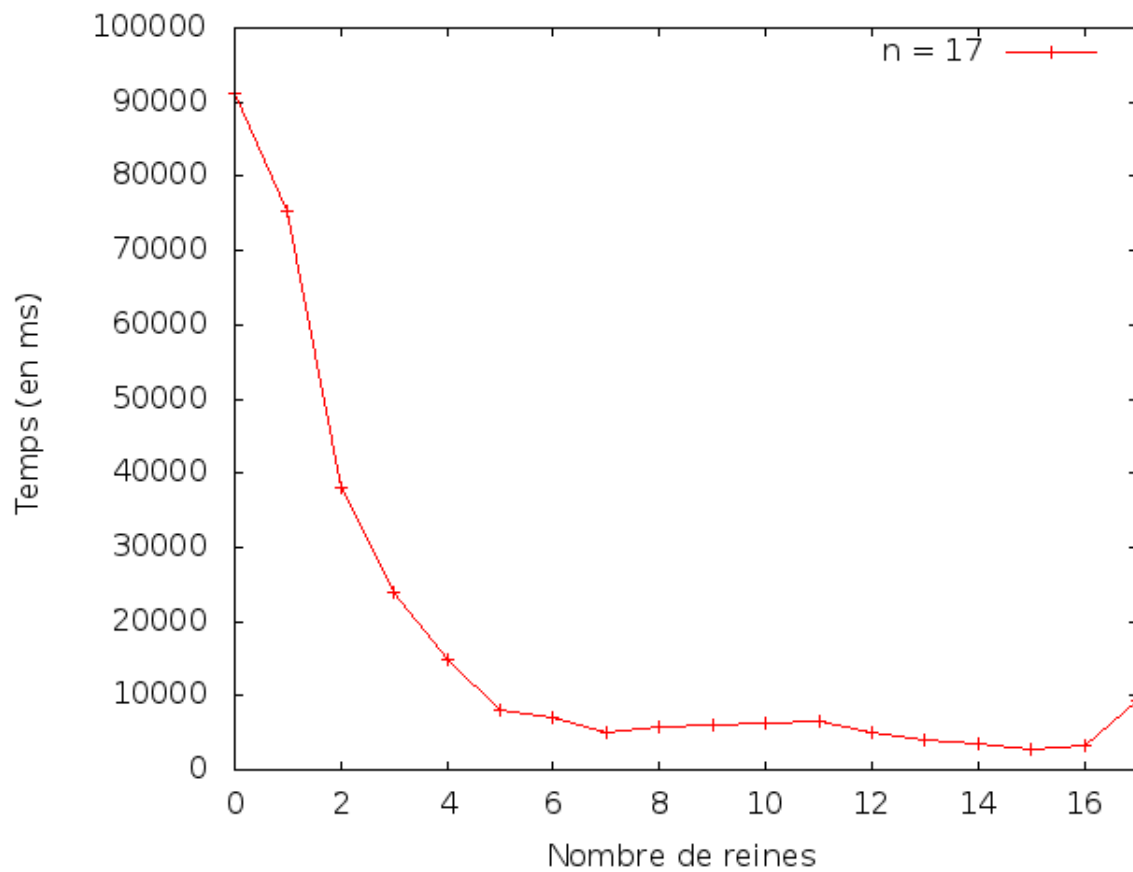


FIGURE 6 – Nombre de combinaisons testées par l’algorithme hybride

Nous observons que l’algorithme devient intéressant seulement pour quelques valeurs de k , et que ces valeurs impliquent une utilisation plus importante de la partie randomisée de l’algorithme que déterministe.

Toutes les solutions

Regardons à présent la capacité de chaque algorithme à résoudre le problème en totalité. Peut-être pourrions nous penser que les résultats obtenus seront proches des précédents. Vérifions...

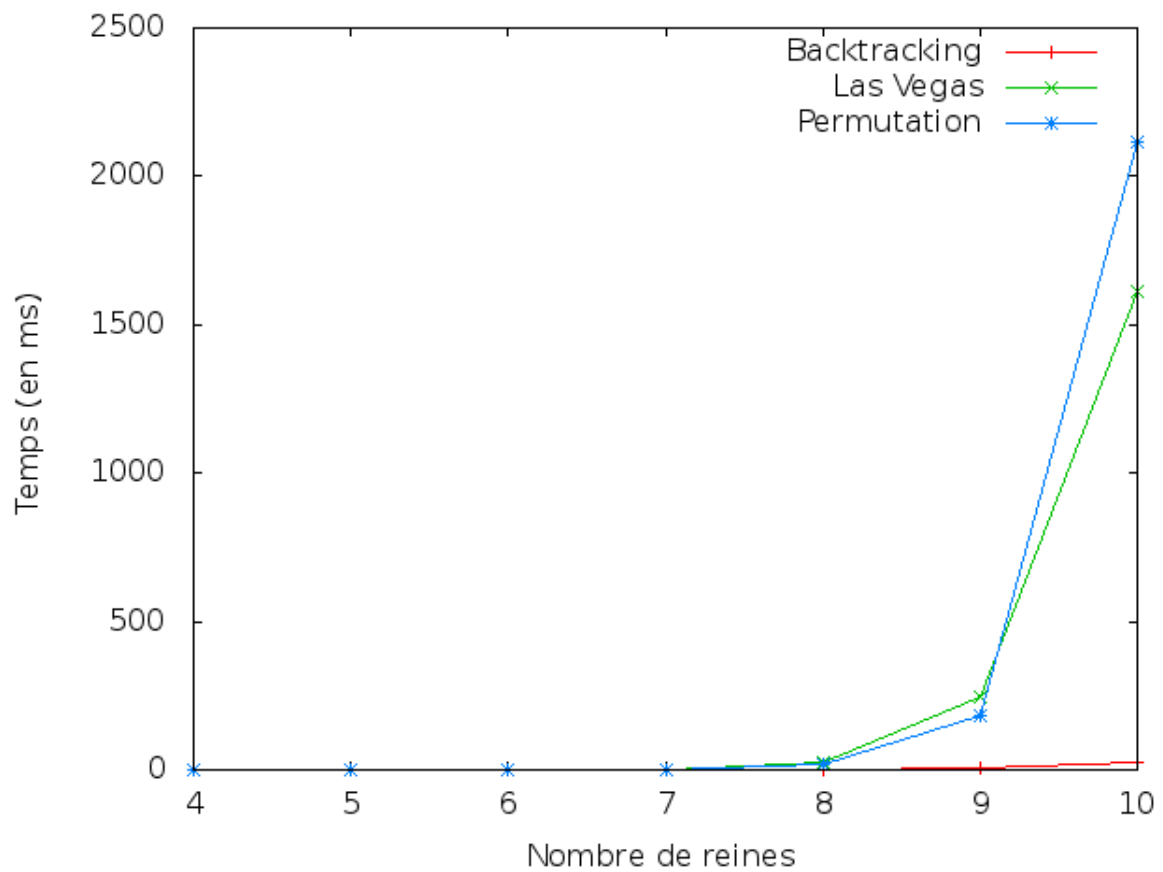


FIGURE 7 – Temps nécessaires à chaque algorithme (sauf hybride)

Ici, je n'ai pas étudié séparément l'algorithme de backtracking et celui de Las Vegas car la figure ci-dessus est assez explicite quant aux efficacités en temps de chaque algorithme. Encore une fois, l'algorithme hybride est traité à part.

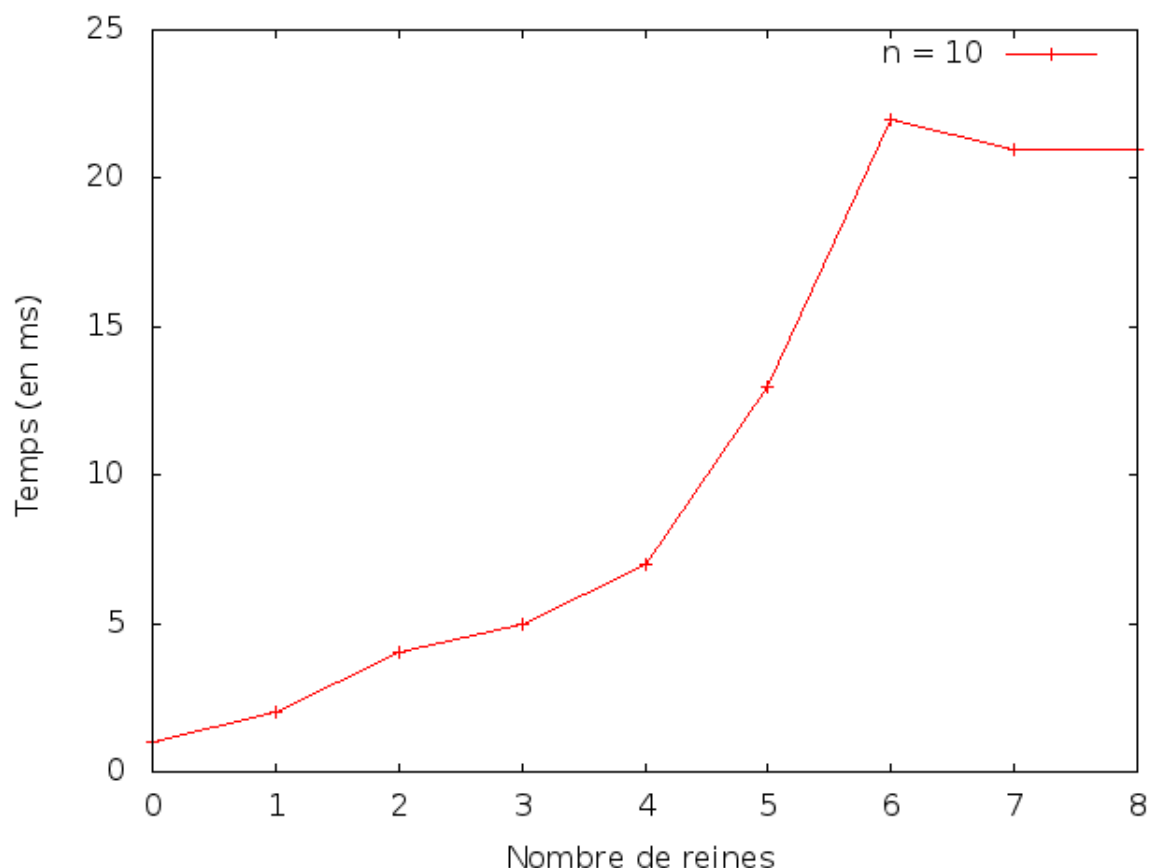


FIGURE 8 – Temps nécessaires à l’algorithme hybride

Nous remarquons que les résultats pour toutes les solutions diffèrent beaucoup de ceux pour la première.

En conclusion

Cette petite étude montre donc que choisir entre algorithme déterministe et randomisé dépend beaucoup du problème considéré. Un algorithme basé sur le hasard (aussi uniforme soit-il) est très performant pour trouver une solution à ce problème, nous voyons bien que ce n’est pas tout à fait la même chose pour les trouver toutes. De plus, comment un tel algorithme (comme Las Vegas) pourrait trouver toutes les solutions tout seul? Un algorithme randomisé le peut difficilement. Il nous a fallu donc, pour ces derniers, utiliser un algorithme déterministe (comme le backtracking) afin de connaître à l’avance le nombre de solutions total. Mais lorsque l’on utilise à bon escient les algorithmes randomisés, nous voyons que le gain est considérable par rapport à ceux déterministes.

Coupler les deux approches s’avère aussi intéressant, en témoigne l’algorithme hybride, cependant nous pouvons noter que le gain ne l’est pas toujours en fonction du moment où l’on transite du randomisé vers le déterministe.